

Scalable Real-time Detection of AI-Generated Arabic Text: A Distributed Data Pipeline Approach

Areej Abdulaziz Alshamani - 4725057

MSBDA-801 Big Data Analytics

Instructor:Dr.Mohammed alsarem

Abstract

This project presents a scalable framework for detecting AI-generated Arabic text using Apache Spark and distributed Big Data processing techniques. The study addresses the growing challenge of distinguishing between human-written and AI-generated Arabic content as modern language models continue to improve text generation quality.

The proposed system integrates distributed Arabic preprocessing, MapReduce-based feature extraction, TF-IDF vectorization, stylometric analysis, batch machine learning, and stream processing within a Spark-based environment executed on CentOS. Arabic preprocessing included normalization, stopword removal, and light stemming, while distributed analytics generated word frequencies, bigrams, and vocabulary richness metrics. Five stylometric features were combined with 5,000-dimensional TF-IDF vectors to construct the final feature representation.

Three machine learning classifiers were evaluated using Spark MLlib: Logistic Regression, Random Forest, and Linear SVM. Experimental results showed that Logistic Regression achieved the best overall performance, reaching approximately 97.6% accuracy and 0.995 ROC-AUC. Additional scalability experiments demonstrated the effectiveness of Spark for handling distributed Arabic NLP workloads and real-time streaming prediction tasks.

The results confirm that combining stylometric features with scalable TF-IDF representations can effectively distinguish AI-generated Arabic text from human-written content within a distributed Big Data environment.

1. Introduction

The widespread adoption of large language models has revolutionized the content industry, enabling the generation of seamless texts for use in academic writing, journalism, and social media platforms.

This raises concerns regarding academic integrity, the spread of misinformation, content credibility, and plagiarism. Given the limited research and linguistic resources available for Arabic compared to English, this presents a significant research challenge.

The motivation behind this project is to develop a comprehensive, efficient, and reliable system for detecting AI-generated Arabic texts by utilizing a wide range of engineered features and advanced machine learning techniques. The primary goal is to bridge this gap by improving Arabic AI-text detection through the precise identification of patterns that distinguish human-written content from AI-generated content.

Big data technologies, such as Apache Spark, provide distributed processing capabilities that enable scalable feature extraction, machine learning, and stream data processing for massive datasets. Integrating big data frameworks with natural language processing techniques allows efficient large-scale processing, analysis, and interpretation of Arabic texts.

The primary objectives of this project are to design and implement a structured, distributed, and scalable workflow capable of detecting AI-generated Arabic abstracts. The project aims to provide insights and representations that support big data analytics and stream processing techniques.

The project focuses on Arabic text preprocessing, scalable feature engineering, AI-generated text classification, and real-time stream data processing

2. Related Work

Recent advances in large language models have increased research interest in AI-generated text detection. Traditional approaches commonly rely on stylometric analysis, TF-IDF representations, and supervised machine learning algorithms such as Logistic Regression and Support Vector Machines. Al-Shaibani and Ahmed [1] demonstrated that stylometric patterns can effectively distinguish AI-generated Arabic text from human-written content.

Arabic NLP research faces additional challenges due to the limited availability of large-scale Arabic datasets and linguistic resources. Previous studies highlighted the importance of scalable preprocessing and feature engineering techniques for improving Arabic text classification performance [2].

Modern AI-generated text systems are mainly based on transformer architectures introduced by Vaswani et al. [3]. Transformer-based models significantly improved text generation quality, making AI-generated text increasingly difficult to detect.

From a Big Data perspective, scalable NLP systems commonly use distributed architectures such as Lambda and Kappa architectures. Lambda Architecture separates processing into batch and real-time layers, while Kappa Architecture focuses primarily on stream processing for low-latency analytics [4].

Apache Spark has become one of the most widely adopted frameworks for distributed NLP and Big Data analytics because it supports scalable machine learning, distributed data processing, and stream processing [5]. Spark NLP libraries further extend Spark capabilities for large-scale natural language processing tasks [6].

This project builds upon these previous works by integrating distributed Arabic preprocessing, MapReduce-based feature extraction, stylometric feature engineering, TF-IDF vectorization, Spark MLlib classification models, and real-time stream processing into a scalable framework for AI-generated Arabic text detection.

3. Dataset Description

- **3.1. Source**

The dataset used in this study was obtained from the HuggingFace repository: [KFUPM-JRCAI Arabic Generated Abstracts Dataset](#). The dataset contains Arabic academic abstracts written by humans along with AI-generated versions produced using multiple large language models.

- **3.2. Size**

The original dataset consists of 8,388 human-written samples. To improve scalability and support distributed processing experiments, additional AI-generated samples were merged during preprocessing to create a larger dataset of 41,940 rows suitable for Spark-based analytics and stream processing evaluation.

- **3.3. Class Distribution**

Binary labels were assigned during preprocessing: human-written abstracts were labelled as 0, and AI-generated abstracts were labelled as 1. The dataset exhibits a significant class imbalance with 8,388 Human samples (20%) and 33,552 AI samples (80%), yielding a 4:1 ratio. This imbalance was addressed during model training via class-weight compensation, assigning higher weight to the minority class (Human) without modifying the underlying data distribution.

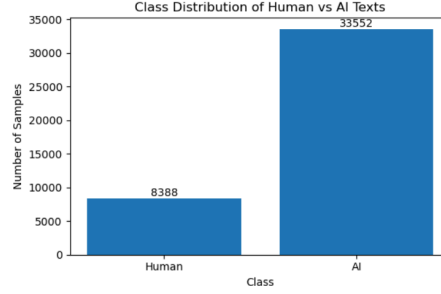


Figure 1: Class distribution

• 3.4. Sample Examples

To better understand the dataset content, random samples were manually inspected. Human-written abstracts generally exhibit formal academic structure, natural stylistic variation, and coherent argumentation. In contrast, AI-generated abstracts closely resemble human writing in structure and length, making manual distinction challenging.

	class	text
0	Human-written	إن محفل أشعار أبي مدين شعيب تعكس تجربته الزهدية التي عبر فيها عما يختلج في نفسه من حب وجمال وقيم أخلاقية ومعركة ومقامات وأحوال يتدرج فيها السالك لبلوغ مرتبة الكشف ، وكان القرآن الكريم المنبع الفياض الذي يستقي منه الشاعر معظم أفكاره وآراءه حيث وظف الفاطمة ومعانيه ومضامينه في أغلب قصائده، ويتجلى ذلك الاقتباس بوضوح في قصيدته البيانية. وبإثباته هذه تحمل توجه الشاعر في إظهار سمات الشوق و الحنين الرجاء و التودد و كلها قيم يرنو إليها الصوفي لبلوغ أسمى درجات التقرب ..
1	AI-generated	تدرس هذه الورقة البحثية تعقيدات التكيف القانوني والعقوبات المرتبطة بحرمة هناك العرض ضمن الإطار القانوني الجزائري. من خلال فحص دقيق للقانون المحلي، تهدف الدراسة إلى تسليط الضوء على التعقيدات والتحديات المرتبطة بتعريف وتصنيف هذه الجريمة، بالإضافة إلى العقوبات المرتبطة بها. النتائج المتوقعة من البحث هي فهم أعمق للآثار العملية لهذه التعقيدات، مما يوفر رؤية قيمة لصانعي السياسات والممارسين القانونيين

Figure 2: Example samples

4. Methodology

• 4.1. Phase 1: Data Acquisition and Storage

The raw dataset was loaded from a local CSV file (raw_combined_abstracts.csv) using Spark with UTF-8 encoding and multiline support. The data was validated through null checks and schema verification, and labels were converted into integer format. The preprocessing and storage pipeline was executed within a CentOS-based distributed analytics environment following Hadoop ecosystem principles. Processed data was stored in Parquet / ORC formats to support scalable distributed analytics.

Initial EDA revealed the label distribution, text length statistics, and null counts. Average character length was 379 for human-written texts and 421 for AI-generated texts. No null values were found after cleaning, resulting in a final dataset of 41,940 rows.

- 4.2. Phase 2: Distributed Preprocessing and MapReduce

Arabic preprocessing was implemented using Spark UDFs and included text normalization, stopwords removal, and light stemming. Normalization removed diacritics, unified Arabic character variations, and filtered non-Arabic symbols. The processed data was saved in Parquet/ORC formats.

Exploratory analysis generated word frequencies, word clouds, bigram analysis, and Type-Token Ratio (TTR). Distributed MapReduce workflows were implemented. These distributed operations conceptually follow the Hadoop MapReduce paradigm for scalable corpus-level aggregation and large-scale text analytics and were executed in a CentOS-based Spark environment. The Map phase generated intermediate key-value pairs such as (word,1) and (bigram,1), while the Reduce phase aggregated frequencies using distributed reduceByKey operations to compute scalable corpus statistics across the dataset.



Figure 3: Word count

Top Unigrams	Frequency	Top Bigrams	Frequency
على	102,309	دراسة إلى	124
إلى	77,732	لغة عربية	124
دراسة	56,578	مشروع جزائري	77
بحث	42,841	ضوء على	76
إن	38,475	ورقة بحثية	70
تحليل	22,672	على مستوى	68
حيث	17,066	إلى أن	68
تأثير	16,477	تهدف دراسة	63
نتائج	16,400	إضافة إلى	59
تعزيز	16,143	إلا أن	54

Figure 4: uni/bi-grams

Vocabulary Richness (TTR)	AI-generated	Human-written
Total Tokens	2,921,194	831,451
Unique Words (Types)	40,429	30,092
TTR Score	0.0138	0.0362

Figure 5: TTR

- 4.3. Phase 3: Feature Engineering and Batch Modeling

Five stylometric features were extracted using distributed Spark UDFs, including digit ratio, punctuation diversity, adjective patterns, genitive structures, and the Gini coefficient of word frequencies. In addition, a MapReduce-style distributed feature extraction workflow was implemented using Spark RDD operations executed in a CentOS environment.

The Map phase extracted intermediate (label, feature_value) pairs from each document, while the Reduce phase aggregated corpus-level statistics using distributed reduceByKey operations. This workflow was used to compute scalable feature statistics for both AI-generated and human-written texts.

TF-IDF representations were generated using Spark MLlib's HashingTF and IDF transformers with 5,000 dimensions. The final feature vector combined TF-IDF features with the five stylometric features, resulting in a 5,005-dimensional sparse vector for each document. The dataset was divided into training (70%), validation (15%), and test (15%) sets. Three Spark MLlib classifiers were trained and evaluated:

→ Logistic Regression (LR) & Random Forest (RF) & Linear (SVM).

```
FEATURE f3: Number of digits / C

M/R Job 1 - Per-document digit count:
MAP   : (doc_id, text) → (doc_id, count_digits(text))
MAP   : (doc_id, text) → (doc_id, len(text))
REDUCE: (doc_id, [digit_count, char_count]) → digit_count / char_count

M/R Job 2 - Corpus-level aggregation:
MAP   : (doc_id, f3_value) → ("f3", f3_value)
REDUCE: ("f3", [v1,v2,...]) → mean(values)
```

Figure 6. Example of the MapReduce workflow

• 4.4. Phase 4: Stream Processing and Scalability

A file-based stream simulation was implemented by continuously generating CSV batch files into a monitored directory. The best-performing model, Logistic Regression, was deployed within the streaming pipeline to perform real-time predictions using the same preprocessing and feature engineering workflow used during batch training.

Comprehensive evaluation included Accuracy, F1-Score, ROC-AUC, and confusion matrices. Scalability experiments were conducted using different Spark resource configurations (local[1], local[2], local[4], and local[8]) to measure execution time, latency, and throughput.

5. Results & Analysis

• 5.1. Model Comparison on Held-out Test Set

Model	Acc	Prec	Rec	F1	AUC
Logistic Regression	0.9760	0.9776	0.9760	0.9764	0.9953
Random Forest	0.9115	0.9317	0.9115	0.9161	0.9792
Linear SVM	0.9745	0.9750	0.9745	0.9746	0.9938

Figure 7: Model comparison

Logistic Regression achieved the best overall performance with $F1=0.9764$ and $ROC-AUC=0.9953$, outperforming both Random Forest ($F1=0.9161$) and Linear SVM ($F1=0.9746$).

- **5.2 Confusion Matrix Analysis**

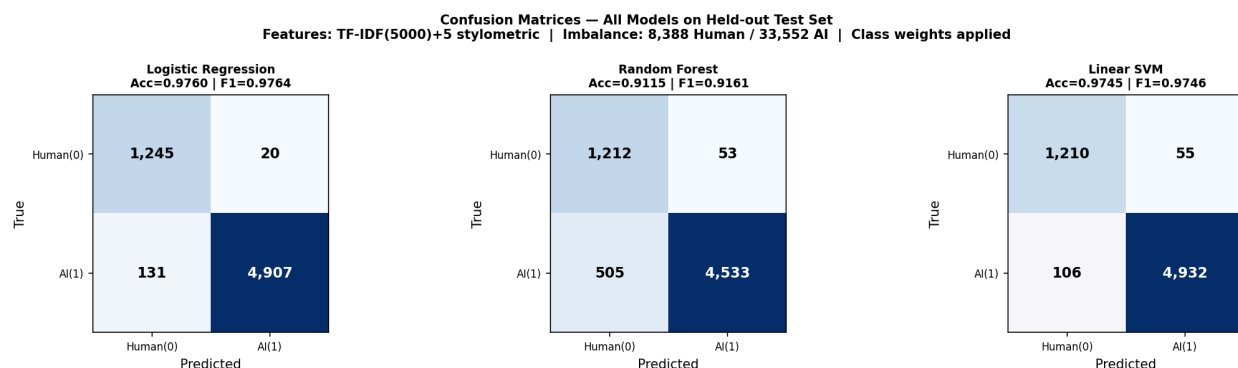


Figure 8: Confusion Metrics

The results show that Logistic Regression and Linear SVM achieved the strongest overall performance, both exceeding 97% accuracy and F1-score, while maintaining balanced predictions for both classes. Logistic Regression achieved the best human-text detection performance with only 20 false positives, whereas Linear SVM slightly reduced false negatives for AI-generated texts.

Random Forest produced lower performance compared to the linear models, particularly in detecting AI-generated abstracts, resulting in a larger number of misclassifications.

Overall, the findings confirm that linear classifiers are more effective for high-dimensional TF-IDF representations and stylometric features in Arabic AI-text detection tasks.

- **5.3 Feature Importance Analysis**

Since Logistic Regression achieved the best overall performance ($F1\text{-score} = 0.9764$), its coefficients were analyzed to interpret the contribution of the assigned stylometric features.

Positive coefficients indicate stronger association with AI-generated text, while negative coefficients indicate stronger association with human-written text.

The results show that **f24 (punctuation diversity)** was the most influential feature with a large negative coefficient (-58.637), indicating that human-written abstracts generally use a richer variety of punctuation marks than AI-generated text.

In contrast, **f3 (digit ratio)** and **f87 (Gini coefficient)** produced positive coefficients, suggesting that higher numeric density and more repetitive word distributions are more associated with AI-generated abstracts.

The remaining features, **f66 (genitive count)** and **f45 (adjective count)**, showed relatively weak influence on the final classification decision.

Figure 9 illustrates both the raw coefficient values and the ranked absolute importance of the five assigned stylometric features.

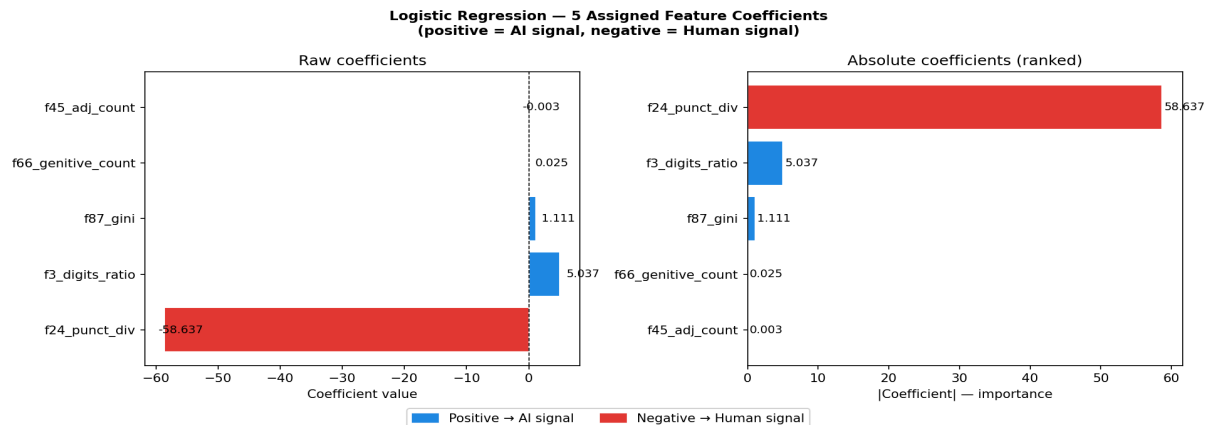


Figure 9: Feature importance analysis

● 5.4. Error Analysis

Error analysis showed that some AI-generated texts were misclassified as human-written when the writing style appeared more formal and diverse. Similarly, a small number of human-written texts were predicted as AI-generated due to repetitive structure and highly organized wording. These observations indicate that stylistic overlap between advanced AI-generated Arabic text and formal academic human writing remains a challenging classification problem.

```
>> Logistic Regression Errors:
=====
True Label : 1.0
prediction : 0.0
ألفظ الطئع والئق في العربية: دراسة في الخلفية الصؤربة

ئعق الجرجاني الطئع بأنه ما يقع على الإنسان بغير إرادة. وئعق الئق بأنه هيئة للنش رلصة صدر عنها الأفعال سهولة وسر من غير حاجة إلى فكر وروية. وئشير ابن
جني إلى ظاهرة لغوية دلالة ئسئ "تلافي المعاني على اختلاف الضؤل والمباني". حيث يجد للمعنى الواحد أسماء كثيرة. وئشير الدراسة إلى ئ الطبع والئق رلصل في ال
نش. ئك

True Label : 0.0
prediction : 1.0
إن مكافحة الجرائم المعلوماتية بطلب لئالب و تطبيق متطورة لحماية المعلوماتية. ولئمكن من الكف عن مرتكبى هذه الأعمال الغير مشروعة وج منح أفراد الشرطة القضا
ئية صلاحيات وسلطات تمكنها من اتخاذ رد فعل نوعى وفعال ضمن صواط قانونية ضمن عدم الحياد عن هذا المسار. و في ذلك الجين فعالية في محاربة هذا النوع من الإجرام ال
خطير و السريع التطور. وفي هذا السياق تأتي هذه الدراسة لمناقشة الإطار الإجرائى الئى نعم
```

Figure 10: Error analysis

- 5.5 Batch vs. Stream Performance

Metric	Batch	Stream
Best throughput (rec/s)	2690	49
Avg latency per batch	~30,000 ms	1125.4 ms
Avg Accuracy	0.9760	0.9120
Avg F1-Score	0.9764	0.9498
Best executor config	local[2]	N/A
Batch size	1,000	50

Table 1: Batch vs. Stream Performance Comparison

Table 1 compares the performance of batch processing (Phase 3) and stream processing (Phase 4) using Apache Spark.

For batch processing, increasing the Spark executors from `local[1]` to `local[2]` improved throughput from 1,779 to 2,690 records/sec, achieving the best overall performance with Accuracy = 0.9760 and F1-score = 0.9764. Using more executors beyond `local[2]` did not consistently improve performance due to parallelization overhead on the local machine.

For stream processing, the Logistic Regression pipeline processed 10 batches of 50 records each, achieving an average latency of 1,125 ms per batch and a throughput of 49 records/sec. The streaming pipeline also maintained stable classification performance with Accuracy = 0.9120 and F1-score = 0.9498.

Overall, Apache Spark successfully supported both batch and real-time stream processing for AI-generated Arabic text detection. Batch processing achieved higher accuracy and throughput because it operated on larger and more stable datasets, while stream processing provided near real-time predictions suitable for live applications.

- 5.6. Scalability Analysis

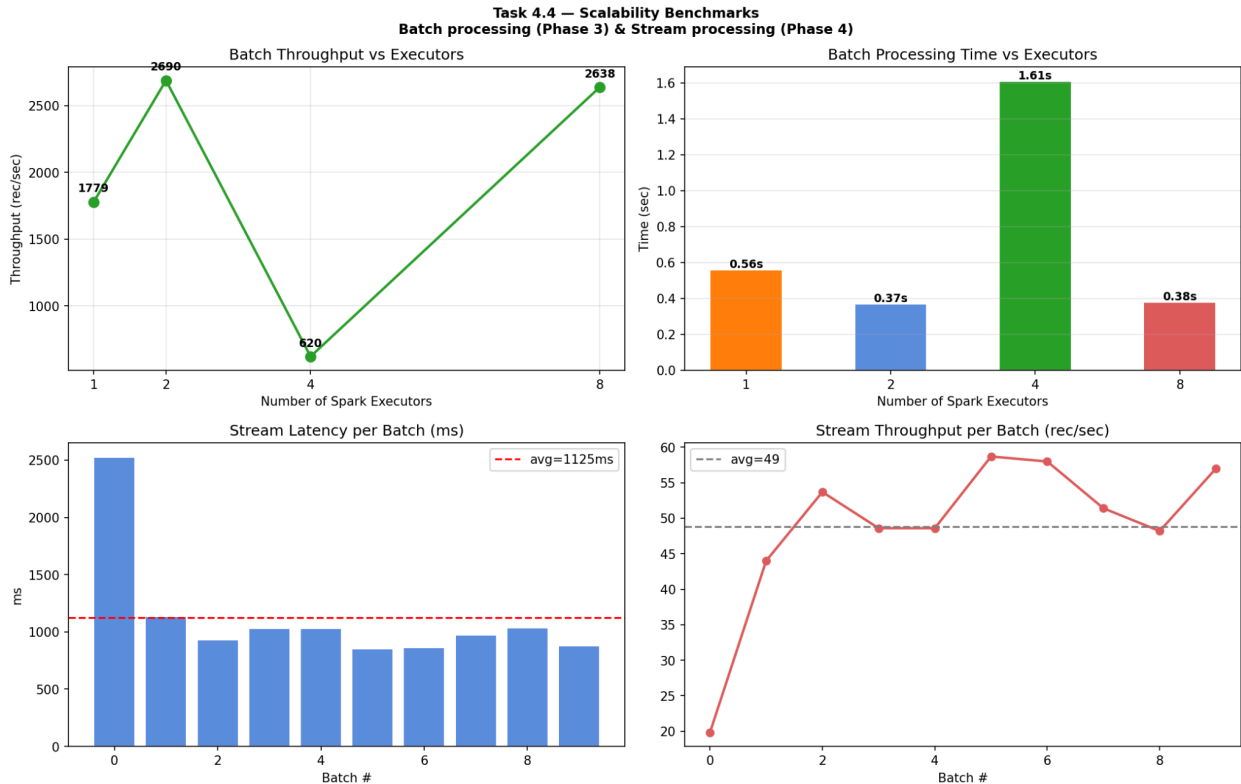


Figure 11: Scalability Benchmarks (Batch & Stream)

The figure contains four charts that summarize the scalability benchmark results.

- ➔ **Top-left — Batch Throughput vs. Executors:** The highest throughput was achieved using 2 executors (2,690 rec/s), which means local[2] gave the best performance on this machine. Using more executors did not always improve performance because extra parallel processing created additional overhead.
- ➔ **Top-right — Batch Processing Time vs. Executors:** The fastest execution time was also achieved at 2 executors (0.37s). When using 4 executors, the execution time became much slower (1.61s), showing that more resources do not always guarantee better performance in local mode.
- ➔ **Bottom-left — Stream Latency per Batch:** The first streaming batch had the highest latency (2,526 ms) because Spark needed time for initialization and loading the model. After that, latency became more stable, with an average of around 1125 ms per batch.
- ➔ **Bottom-right — Stream Throughput per Batch:** The streaming system processed about 49 records/sec on average. Small differences between batches appeared because each stream batch contained only 50 records.

6. Conclusion & Future Work

This project successfully developed a scalable Arabic AI-generated text detection framework using Apache Spark and distributed processing techniques. The system combined distributed preprocessing, MapReduce-style feature extraction, TF-IDF vectorization, stylometric analysis, and machine learning classification within a unified Big Data pipeline.

The experimental results demonstrated that Logistic Regression achieved the best performance among the evaluated models, with high accuracy and ROC-AUC scores. Error analysis also showed that most misclassifications occurred in highly formal academic texts where stylistic similarities between AI-generated and human-written Arabic content became difficult to distinguish.

The project additionally demonstrated the practical integration of batch processing, stream processing, and scalable analytics using Spark within a CentOS-based environment following Hadoop ecosystem principles.

Despite the strong performance, several limitations remain. The dataset focused mainly on Arabic academic-style texts, which may limit generalization to other Arabic domains such as social media or informal dialects. In addition, the system relied primarily on traditional machine learning and stylometric features rather than deep neural language models.

- 6.1. Challenges and Limitations

Some practical challenges were faced during the project implementation. Spark initially showed compatibility and environment issues on CentOS, so the experiments were later moved to Spark on Windows for more stable execution.

Another challenge was using Spark Python UDFs for Arabic preprocessing and feature extraction. Although flexible, UDFs were slower and introduced additional overhead due to communication between Python and the Spark JVM.

Kafka was also considered for stream simulation, but Kafka version 4.1.2 lacked Windows startup scripts, which complicated deployment on Windows. Therefore, a file-based streaming simulation approach was used instead.

- 6.2. Future work

Future improvements may include using larger Arabic datasets, testing deep learning models such as AraBERT, and integrating Kafka-based real-time streaming instead of file-based simulation. Additional Arabic linguistic features and more scalable cloud deployment environments could also improve system performance and real-time processing capabilities.

References

- [1] M. S. Al-Shaibani and M. Ahmed, “The Arabic AI Fingerprint: Stylometric Analysis and Detection of Large Language Models Text,” *arXiv preprint arXiv:2505.23276*, 2025.
- [2] E. Boujou, A. Chouigui, and A. Dahou, “An Open Access NLP Dataset for Arabic Dialects: Data Collection, Labeling, and Model Construction,” *arXiv preprint arXiv:2102.11000*, 2021.
- [3] A. Vaswani *et al.*, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.
- [4] J. Kreps, “Questioning the Lambda Architecture,” *O’Reilly Radar*, 2014.
- [5] M. Zaharia *et al.*, “Apache Spark: A Unified Engine for Big Data Processing,” *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [6] John Snow Labs, “Spark NLP: Natural Language Processing at Scale,” GitHub Repository, 2025. Available: <https://github.com/JohnSnowLabs/spark-nlp>

Appendix:

HDFS Project Implementation Screenshots:

```
hadoop@aree:~$ hdfs dfs -ls /bigdata_project
2026-05-20 11:33:06,683 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 1 items
drwxr-xr-x 1 hadoop supergroup 0 2026-05-20 11:31 /bigdata_project/bigdata_project
hadoop@aree:~$ hdfs dfs -ls /bigdata_project/bigdata_project
2026-05-20 11:34:18,075 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 25 items
-rw-r--r-- 1 hadoop supergroup 59 2026-05-20 11:30 /bigdata_project/bigdata_project/.gitignore
drwxr-xr-x 1 hadoop supergroup 0 2026-05-20 11:30 /bigdata_project/bigdata_project/data
-rw-r--r-- 1 hadoop supergroup 8762 2026-05-20 11:30 /bigdata_project/bigdata_project/extract_features_dataset.py
-rw-r--r-- 1 hadoop supergroup 4979 2026-05-20 11:30 /bigdata_project/bigdata_project/logs_features.txt
-rw-r--r-- 1 hadoop supergroup 5491 2026-05-20 11:30 /bigdata_project/bigdata_project/logs_phase1.txt
-rw-r--r-- 1 hadoop supergroup 33292 2026-05-20 11:30 /bigdata_project/bigdata_project/logs_phase2.txt
-rw-r--r-- 1 hadoop supergroup 11161 2026-05-20 11:30 /bigdata_project/bigdata_project/logs_phase3.txt
-rw-r--r-- 1 hadoop supergroup 9429 2026-05-20 11:30 /bigdata_project/bigdata_project/logs_phase3_v2.txt
-rw-r--r-- 1 hadoop supergroup 11384 2026-05-20 11:30 /bigdata_project/bigdata_project/logs_phase3_v3.txt
-rw-r--r-- 1 hadoop supergroup 11545 2026-05-20 11:31 /bigdata_project/bigdata_project/logs_phase4_final.txt
-rw-r--r-- 1 hadoop supergroup 15020 2026-05-20 11:30 /bigdata_project/bigdata_project/logs_phase4_v2.txt
-rw-r--r-- 1 hadoop supergroup 7306 2026-05-20 11:31 /bigdata_project/bigdata_project/logs_phase4_v3.txt
-rw-r--r-- 1 hadoop supergroup 16469 2026-05-20 11:30 /bigdata_project/bigdata_project/mapreduce_features.py
drwxr-xr-x 1 hadoop supergroup 0 2026-05-20 11:30 /bigdata_project/bigdata_project/models
drwxr-xr-x 1 hadoop supergroup 0 2026-05-20 11:30 /bigdata_project/bigdata_project/notebooks
-rw-r--r-- 1 hadoop supergroup 6087 2026-05-20 11:30 /bigdata_project/bigdata_project/phase1_setup.py
-rw-r--r-- 1 hadoop supergroup 11254 2026-05-20 11:30 /bigdata_project/bigdata_project/phase2_preprocessing.py
-rw-r--r-- 1 hadoop supergroup 15744 2026-05-20 11:30 /bigdata_project/bigdata_project/phase3_feature_engineering.py
-rw-r--r-- 1 hadoop supergroup 26211 2026-05-20 11:30 /bigdata_project/bigdata_project/phase4_streaming.py
-rw-r--r-- 1 hadoop supergroup 1756 2026-05-20 11:31 /bigdata_project/bigdata_project/plot_lr_coefficients.py
drwxr-xr-x 1 hadoop supergroup 0 2026-05-20 11:30 /bigdata_project/bigdata_project/reports
-rw-r--r-- 1 hadoop supergroup 104 2026-05-20 11:30 /bigdata_project/bigdata_project/requirements.txt
drwxr-xr-x 1 hadoop supergroup 0 2026-05-20 11:30 /bigdata_project/bigdata_project/src
drwxr-xr-x 1 hadoop supergroup 0 2026-05-20 11:31 /bigdata_project/bigdata_project/stream
-rw-r--r-- 1 hadoop supergroup 3802 2026-05-20 11:31 /bigdata_project/bigdata_project/task44_compare.py
hadoop@aree:~$
```

Figure A1: HDFS project structure.